



You can view this report online at : <https://www.hackerrank.com/x/tests/190608/candidates/26629279/report>

Full Name: Akhil Tiwari
Email: akhiltiwari.13@gmail.com
Test Name: Optiver Software Engineer (48h)
Taken On: 13 Jun 2021 09:30:34 CEST
Time Taken: 2750 min 41 sec/ 2880 min
Invited by: Patrycja
Invited on: 7 Jun 2021 13:46:18 CEST
Skills Score:
Tags Score:

Algorithms	3/40
Difficult	70/100
Easy	20/25
Medium	13/55
Strings	3/40
coding	85/115
theory	15/25

57.2%
103/180

scored in **Optiver Software Engineer (48h)** in 2750 min 41 sec on 13 Jun 2021 09:30:34 CEST

Recruiter/Team Comments:

No Comments.

	Question Description	Time Taken	Score	Status
Q1	Virtual Functions > Multiple Choice	2 min	0/ 5	✗
Q2	Abstract Functions > Multiple Choice	4 min 19 sec	5/ 5	✓
Q3	Shapes > Coding	22 min 59 sec	15/ 15	✓
Q4	Data structures - Financial instruments > Multiple Choice	2 min 50 sec	5/ 5	✓
Q5	Data Structures - City to Surf race > Multiple Choice	5 min 41 sec	5/ 5	✓
Q6	Time complexity - CalculateAverageFromTable > Multiple Choice	1 min 45 sec	0/ 5	✗
Q7	Royal Names > Coding	11 hour 22 min 21 sec	3/ 40	✓
Q8	Travelling the graphs > Coding	16 hour 32 min 5 sec	70/ 100	✓

QUESTION 1

Wrong Answer

Score 0

Virtual Functions > Multiple Choice

Easytheory

QUESTION DESCRIPTION

A virtual function... (*all functions in Java should be considered virtual by default*)

CANDIDATE ANSWER

Options: (Expected answer indicated with a tick)

☒ ☐ can be re-implemented in a derived class

☒ ☐ is a function for which different derived classes can have different implementations

☐ is a function for which a derived class must provide a new implementation

☐ ☒ is a function whose existence in a class will prevent instantiation of that class

No Comments

QUESTION 2

Correct Answer

Score 5

Abstract Functions > Multiple Choice

Easytheory

QUESTION DESCRIPTION

Abstract functions... (*pure virtual in C++ terminology*)

CANDIDATE ANSWER

Options: (Expected answer indicated with a tick)

☒ ☐ prevent instantiation of that class

☐ can exist in non-abstract classes

☐ can only have one implementation

☒ ☐ are a way to define a contract

No Comments

QUESTION 3

Correct Answer

Score 15

Shapes > Coding

Easycoding

QUESTION DESCRIPTION

Start out by creating an interface Shape, which has an abstract function GetArea() with integer return value. Then create implementations of this interface for three different shapes: Rectangle, Triangle and Circle.

The formulas for calculating the area of each of these shapes are:

Rectangle: height x width
Triangle: (height x width) / 2
Circle: 3.14 x radius x radius

The integer values required to calculate the area for each of these shapes will be provided to the constructor of each shape respectively in the order of appearance in the examples below.

Round the result of the calculation to the nearest integer value before returning it.

Examples

Rectangle(height = 4, width = 3) : GetArea() returns 12

Triangle(height = 5, width = 2) : GetArea() returns 5

Circle(radius = 5) : GetArea() returns 79

The total area of these shapes is 96.

CANDIDATE ANSWER

Language used: **C++14**

```
1 #include <cmath>
2
3 class Shape{
4     public:
5     virtual int GetArea()=0;
6 };
7
8 class Rectangle: public Shape{
9     public:
10    Rectangle(int height, int width):height(height), width(width){
11    }
12
13    int GetArea(){
14        return height*width;
15    }
16
17    private:
18    int height;
19    int width;
20 };
21
22 class Triangle: public Shape{
23     public:
24    Triangle(int height, int width):height(height), width(width){
25    }
26
27    int GetArea(){
28        auto area = (static_cast<double>(height)*static_cast<double>
29 (width))/2.0;
30        return round(area);
31    }
32
33    private:
34    int height;
35    int width;
36 };
37
38 class Circle: public Shape{
39     public:
40    Circle(int radius):radius(radius){
41    }
42
43    int GetArea(){
44        auto area = static_cast<double>(pow(radius,2)) *3.14 ;
45        return round(area);
46    }
47 }
```

```
48 private:
49     int radius;
50 };
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
> Example	Easy	Sample case	 Success	5	0.0204 sec	9.01 KB
Test case 1 (Triangle area rounds up)	Easy	Hidden case	 Success	5	0.02 sec	8.92 KB
Testcase 2 (Circle area rounds down)	Easy	Hidden case	 Success	5	0.0146 sec	8.96 KB

No Comments

QUESTION 4



Correct Answer

Score 5

Data structures - Financial instruments > Multiple Choice

theory

Medium

QUESTION DESCRIPTION

A certain program must store a very large collection of names of financial instruments. The user can enter these names in any order, and can query if a name already exists in the collection. Each name must appear in the collection only once. The program also has a feature which prints out all of the names in no specific order when required.

Which data structure would be the best choice for this program?

CANDIDATE ANSWER

Options: (Expected answer indicated with a tick)

- ☐ A vector or resizable array
-  ☒ A hash set (implemented as a hash table of keys)
- ☐ An array
- ☐ A singly linked list

No Comments

QUESTION 5



Correct Answer

Score 5

Data Structures - City to Surf race > Multiple Choice

theory

Medium

QUESTION DESCRIPTION

A certain program must store the names of all the people who finish the City to Surf race. The names are entered by an official at the finish line as the competitors finish. After the race is over the program must print a list only containing the names of competitors who placed 100th, 200th, 300th, and so on, as these competitors will receive a special medallion.

Which data structure would be the best choice for this program?

CANDIDATE ANSWER

Options: (Expected answer indicated with a tick)



A vector or resizable array



A hash set (implemented as a hash table of keys)



A map (implemented as a binary tree of key/value pairs)



A singly linked list

No Comments

QUESTION 6



Wrong Answer

Score 0

Time complexity - CalculateAverageFromTable > Multiple Choice

theory

Medium

QUESTION DESCRIPTION

```
function CalculateAverageFromTable(values, total_rows, total_columns):  
    sum = 0  
    n = 0  
  
    for y from 0 to total_rows:  
        for x from 0 to total_columns:  
            sum += values[y][x]  
            n += 1  
  
    return sum / n
```

Which of the below answers describes the time complexity of the above code most accurately?

CANDIDATE ANSWER

Options: (Expected answer indicated with a tick)



$O(n!)$



$O(n^2)$



$O(n)$



$O(n \log n)$

No Comments

QUESTION 7



Royal Names > Coding

Medium

Strings

Algorithms

Score 3

An *ordinal number* is a word representing rank or sequential order. The naming convention for *royal names* is to follow a *first name* with an *ordinal number*, which is essentially a Roman numeral used to indicate the birth order of two people having the same name. The Roman numerals from 1 to 50 are defined as follows:

- The respective numerals corresponding to numbers 1 through 10 are *I, II, III, IV, V, VI, VII, VIII, IX*, and *X*.
- The respective numerals corresponding to the numbers 20, 30, 40, and 50 are *XX, XXX, XL*, and *L*.
- The numeral for any other two-digit number < 50 is constructed by concatenating the numeral(s) for its multiples of ten with the numeral(s) for its values < 10. For example, 47 is $40 + 7 = \text{"XL"} + \text{"VII"} = \text{"XLVII"}$.

Complete the *getSortedList* function in your editor. It has 1 parameter: an array of royal name strings, *names*. Each royal name string consists of a *first name*, followed by a single space, followed by a *Roman numeral*. Your function must sort the *names* lexicographically (alphabetically) by first name; if two or more first names are the same, it must sort those duplicate first names by ascending ordinal number. It must then return the sorted array of royal name strings.

Input Format

The locked stub code in your editor reads the following input from stdin and passes it to your function: The first line contains an integer, *n* (the number of elements in *names*). Each line *i* of the *n* subsequent lines contains a string describing royal name *i* (where $0 \leq i < n$) in the *names* array.

Constraints

- $1 \leq n \leq 50$
- Each *names[i]* (where $0 \leq i < n$) is a single string composed of 2 space-separated values: *firstName* and *ordinal*, respectively. Here, *firstName* is the first name and *ordinal* is a valid Roman numeral representing a number between 1 and 50, inclusive.
- $1 \leq |firstName| \leq 20$
- Each *firstName* starts with an uppercase letter (A – Z) and its remaining characters are lowercase letters (a – z).
- Each *ordinal* is a Roman numeral containing the capital letters *I, V, X*, and/or *L*.
- The elements in *names* are pairwise distinct.

Output Format

Your function must return a sorted array of royal name strings. This is printed to stdout by the locked stub code in your editor.

Sample Input 0

```
2
Louis IX
Louis VIII
```

Sample Output 0

```
Louis VIII
Louis IX
```

Sample Input 1

```
2
Philippe I
Philip II
```

Sample Output 1

```
Philip II
Philippe I
```

Explanation

Recall that names take precedence over ordinal numbers. This means that the royal names must be sorted lexicographically by name, and then each subset of non-unique names must be sorted by ordinal number.

Sample Case 0:

Because both names are the same, we only need to sort by ordinal here. The ordinal number *VIII* (8) comes before the ordinal number *IX* (9), so our sorted array must order *Louis VIII* before *Louis IX*.

Sample Case 1:

When sorted lexicographically, *Philip* comes before *Philippe*. Because both names are unique, we do not need to further sort by ordinal. Thus, our sorted array orders *Philip II* before *Philippe I*.

CANDIDATE ANSWER

Language used: **C++14**

```
1 int romanToDecimal(const string& str){
2     unordered_map<char, int> dict{ {'I',1}, {'V',5}, {'X',10}, {'L',50}};
3     int result = dict[str.back()];
4
5     for(int i=(str.size()-2); i>=0; i--){
6         if(dict[i] < dict[i+1]){
7             result -= dict[i];
8         }else{
9             result += dict[i];
10        }
11    }
12
13    return result;
14 }
15
16 // Complete the getSortedList function below.
17 vector<string> getSortedList(const vector<string>& names) {
18
19     auto customComp =[](string s1, string s2)->bool{
20         string fname1= s1.substr(0, s1.find(" ") -1);
21         string fname2= s2.substr(0, s2.find(" ") -1);
22
23         string lname1= s1.substr(s1.find(" ")+1);
24         string lname2= s2.substr(s2.find(" ")+1);
25
26         if(fname1 == fname2) return romanToDecimal(lname1) <
27 romanToDecimal(lname2);
28         else return fname1 < fname2;
29     };
30
31     set <string, decltype(customComp)> snames(begin(names),end(names),
32 customComp);
33     return vector<string>(snames.begin(), snames.end());
34 }
35
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
> Same name, different number	Easy	Sample case	 Success	1	0.0161 sec	8.95 KB

> Different names	Easy	Sample case	✓ Success	1	0.0183 sec	9.05 KB
> Partially matching names	Easy	Sample case	✓ Success	1	0.0164 sec	9.03 KB
Mixed with simple numbers	Easy	Hidden case	✗ Wrong Answer	0	0.0178 sec	9.04 KB
Mixed with bigger numbers	Medium	Hidden case	✗ Wrong Answer	0	0.0153 sec	9.12 KB
Mixed with bigger numbers (2)	Medium	Hidden case	✗ Wrong Answer	0	0.014 sec	8.92 KB
Mixed with bigger numbers (3)	Medium	Hidden case	✗ Wrong Answer	0	0.0154 sec	8.95 KB
Mixed with bigger numbers (4)	Medium	Hidden case	✗ Wrong Answer	0	0.0186 sec	9.09 KB
Mixed with bigger numbers (5)	Medium	Hidden case	✗ Wrong Answer	0	0.0154 sec	8.9 KB
Mixed with bigger numbers (6)	Medium	Hidden case	✗ Wrong Answer	0	0.0158 sec	8.98 KB

No Comments

QUESTION 8



Correct Answer

Score 70

Travelling the graphs > Coding

coding

Difficult

QUESTION DESCRIPTION

You are given a graph written as a sequence of edges with associated distance (travel time), as well as a start and destination node with an associated maximum allowed travel time. All edges can be traveled both ways.

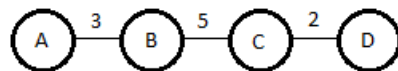
Determine the single shortest path between the start and destination, and print the route if the distance of the path is equal or less than the maximum allowed travel time.

Input Format

Input is read from standard input and has the following characteristics:

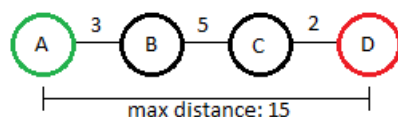
- It is two lines; the first line describes the edges of the graph,
 - Each edge is formatted as an opening bracket '[', the first node of the pair, a comma, the second node in the pair, another comma, the distance between the nodes, followed by a closing bracket ']'. Example: '[A,B,5]'

First line: [A,B,3] [B,C,5] [C,D,2]



- the second line describes the start and destination node, as well as the maximum allowed travel time.
 - the starting node, followed by an ascii arrow '->' (two characters), the destination node, a comma, and the maximum allowed travel time. Example 'A->D,5'

Second line: A->D,15



- Leading or trailing whitespace is not allowed.
- All nodes are single, uppercase letters.

- Edges are separated by a single space.
- Distances are specified as unsigned integers.
- The sequence of edges is not ordered in any specific way.

Output

Output is written to standard output and must have the following characteristics:

- It is one line.
- It contains no whitespace.
- If errors are present, print the first of the below listed errors (e.g. if both E2 and E3 are present, print "E2").
- If no errors are present, print the route using the representation as described below.

Errors

You should detect the following errors:

<i>Code</i>	<i>Type</i>
E1	Input syntax error
E2	Logical input error
E3	Failed to find a suitable route

Logical input errors are; duplicate definitions of edges, specifying either a start or destination node (second line) that is not actually defined in the graph (first line), disconnected graphs, or more than one shortest path found.

Output Representation

If there were no errors and the shortest route is found, the nodes should be printed in traveling order, including the start and destination, separated by ascii arrows '->' (two characters).

Sample Input #00

```
[A,B,3] [B,C,5] [C,D,2]
A->D,10
```

Sample Output #00

```
A->B->C->D
```

Sample Input #01

```
[A,B,3] [A,C,7] [C,D,2] [B,C,5]
A->D,10
```

Sample Output #01

```
A->C->D
```

Sample Input #02

```
[A,B,5] [A,C,2] [B,C,4] [B,D,6] [C,B,7]
A->C,10
```

Sample Output #02

```
E2
```

Output #02 Explanation

The last edge [C,B,7] on the line is a re-definition (duplication) of a previously defined edge [B,C,4]. Such duplication should result in a logical input error.

CANDIDATE ANSWER

Language used: **C++14**

```
1  #include <map>
2  #include <set>
3  #include <list>
4  #include <cmath>
5  #include <ctime>
6  #include <deque>
7  #include <queue>
8  #include <stack>
9  #include <string>
10 #include <bitset>
11 #include <cstdio>
12 #include <limits>
13 #include <vector>
14 #include <climits>
15 #include <cstring>
16 #include <cstdlib>
17 #include <fstream>
18 #include <numeric>
19 #include <sstream>
20 #include <iostream>
21 #include <algorithm>
22 #include <unordered_map>
23
24 /* added header. */
25 #include <regex>
26
27 using namespace std;
28
29 namespace mysol {
30     class Sol {
31     public:
32         void ParseLine(const string &line, unsigned int lineCount) {
33             switch (lineCount) {
34                 case 0: {
35                     regex pattern("[\\]([A-Z]),([A-Z]),(\\d+)"
36 [\\])");
37                     const string delim(" ");
38                     string::size_type begIdx{}, endIdx{};
39                     begIdx = line.find_first_not_of(delim);
40
41                     while (begIdx != string::npos) {
42                         endIdx = line.find_first_of(delim,
43 begIdx);
44
45                     if (endIdx == string::npos) {
46                         // end of word is end of line
47                         endIdx = line.length();
48                     }
49
50                     auto pMatch =
51 hasProperFormat(line.substr(begIdx, (endIdx - begIdx)), pattern);
52                     if (pMatch.first) {
53                         constructGraph(pMatch.second);
54                     }
```

```

55         cout << "E1" << std::endl;
56         exit(0);
57     }
58
59     begIdx = line.find_first_not_of(delim,
60 endIdx);
61     }
62
63     } break;
64     case 1: {
65         regex pattern("[A-Z]->([A-Z]),(\\d+)");
66         smatch match;
67         bool mresult = regex_match(line, match,
68 pattern);
69         if (!mresult) {
70             cout << "E1" << std::endl;
71             exit(0);
72         } else {
73             source = match[1].str()[0];
74             destination = match[2].str()[0];
75             maxAllowedTime = stoi(match[3].str());
76
77             if(!adjacencyList.count(source) ||
78 !adjacencyList.count(destination)){
79                 cout << "E2" << std::endl;
80                 exit(0);
81             }
82         }
83     } break;
84     default: {
85         cout << "E1" << endl;
86         exit(0);
87     } break;
88 }
89 }
90
91 void FindShortestPath(){
92
93     /* auxiliary data-structure for shortest path algorithm
94 (Dijkstras) */
95     unordered_map<char,unsigned int> minTime;
96     unordered_map<char,char> visitedFrom;
97     priority_queue<pair<unsigned int,char>> q;
98
99     /* initializations. */
100     q.push(make_pair(0, source));
101     for(auto n: adjacencyList){
102         minTime[n.first] = std::numeric_limits<unsigned
103 int>::max();
104     }
105     minTime[source] = 0;
106     visitedFrom[source] = '\\0';
107
108     while(!q.empty()){
109         char vert = q.top().second;
110         q.pop();
111
112         //visit each neighbour of the node from the adjacency
113 list.
114         for(auto edge: adjacencyList[vert]){
115             auto neighbour = edge.first;
116             auto wt = edge.second;
117
118             if((minTime[vert]+wt) < minTime[neighbour]){

```

```

11         minTime[neighbour] = minTime[vert] + wt;
12         q.push(make_pair(minTime[neighbour], neighbour));
13         visitedFrom[neighbour] = vert;
14     }
15 }
16
17 auto result = minTime[destination];
18 if(result <= maxAllowedTime){
19
20     /* @todo- print route. */
21     std::vector<char> path;
22
23     char nd = destination;
24     path.push_back(destination);
25     while(nd != source){
26         path.push_back(visitedFrom[nd]);
27         nd=visitedFrom[nd];
28     }
29     reverse(path.begin(), path.end());
30     for(auto n: path){
31         cout<<n;
32         if(n!=destination)cout<<"-";
33     }
34     std::cout<<std::endl;
35 }else{
36     std::cout<<"E3"<<std::endl;
37 }
38
39 private:
40     // graph representation as adjacency list.
41     unordered_map<char, unordered_map<char, unsigned int> >
42 adjacencyList;
43     char source, destination;
44     unsigned int maxAllowedTime;
45
46     void out(bool b) { cout << (b ? "found" : "not found") << endl; }
47
48     pair<bool, smatch> hasProperFormat(const string &str, regex
49 &pattern) {
50
51         smatch match;
52         bool result = regex_match(str, match, pattern);
53
54         return make_pair(result, match);
55     }
56
57     void constructGraph(smatch &match) {
58         auto sourceNode = match[1].str()[0];
59         auto destinationNode = match[2].str()[0];
60         unsigned int weight = stoul(match[3].str());
61
62         if(adjacencyList[sourceNode].count(destinationNode)){
63             cout << "E2" << std::endl;
64             exit(0);
65         }else{
66             adjacencyList[sourceNode].insert(make_pair(destinationNode, weight));
67             adjacencyList[destinationNode].insert(make_pair(sourceNode, weight));
68         }
69     }
70 }

```

```

18 } // namespace mysol
18
18
18
6 int main() {
    mysol::Sol sol;
    string line{};

    for(int i=0; i<2; ++i)
    {
        getline(cin, line);
        sol.ParseLine(line, i);
    }

    sol.FindShortestPath();
    return 0;
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
> Testcase 0 - Sample 1	Easy	Sample case	✓ Success	2	0.019 sec	9.13 KB
> Testcase 1 - Sample 2	Medium	Sample case	✓ Success	4	0.0153 sec	9.13 KB
> Testcase 2 - Sample 3	Medium	Sample case	✓ Success	4	0.0136 sec	8.98 KB
Testcase 3 - Straight path, start and destination not leaves	Easy	Hidden case	✓ Success	3	0.0133 sec	9.13 KB
Testcase 4 - Straight path, more complex graph	Medium	Hidden case	✓ Success	6	0.0155 sec	9.13 KB
> Testcase 5 - Multiple paths available 1	Medium	Sample case	✓ Success	6	0.0232 sec	9.04 KB
Testcase 6 - Multiple paths available 2	Medium	Hidden case	✓ Success	8	0.0159 sec	9.13 KB
> Testcase 7 - Syntax error 1: Non-alphabetical nodes	Easy	Sample case	✓ Success	3	0.0164 sec	9.03 KB
> Testcase 8 - Syntax error 2: Curly brackets are prettier, aren't they?	Easy	Sample case	✓ Success	3	0.0187 sec	9.19 KB
Testcase 9 - Syntax error 3: Delimiters replaced by spaces	Easy	Hidden case	✓ Success	3	0.017 sec	9.03 KB
Testcase 10 - Syntax error 4: Error on second line	Medium	Hidden case	✓ Success	8	0.0169 sec	9.21 KB
Testcase 11 - Logical error 1: Start not in graph	Easy	Hidden case	✓ Success	3	0.0138 sec	8.95 KB
> Testcase 12 - Logical error 2: Destination not in graph	Easy	Sample case	✓ Success	3	0.0139 sec	9.02 KB
Testcase 13 - Logical error 3: Duplicate edges specified 1	Easy	Hidden case	✓ Success	3	0.017 sec	8.99 KB
> Testcase 14 - Logical error 4: Duplicate edges specified 2	Medium	Sample case	✓ Success	5	0.0166 sec	9.18 KB
Testcase 15 - Disconnected graphs (requested route available)	Hard	Hidden case	✗ Wrong Answer	0	0.0243 sec	9.04 KB
Testcase 16 - Logical error 6: Multiple equal routes available	Hard	Hidden case	✗ Wrong Answer	0	0.0158 sec	9.05 KB

> Testcase 17 - Failed to find a suitable route 1	Medium	Sample case	 Success	6	0.0155 sec	9.13 KB
Testcase 18 - Disconnected graphs (requested route available but too far)	Hard	Hidden case	 Wrong Answer	0	0.0213 sec	8.99 KB

No Comments

PDF generated at: 15 Jun 2021 07:25:33 UTC